

# Dynamic UAV Routing Under Distribution Shift: A Reproducible Multi-Seed Comparison of Reinforcement Learning and Classical Search

Simra Imran\* and Muzzammil Sajid†

\*School of Electrical Engineering and Computer Science, National University of Sciences and Technology  
Islamabad, Pakistan — simraimran158@gmail.com

†Faculty of Mechanical Engineering, Ghulam Ishaq Khan Institute of Engineering Sciences and Technology  
Topi, Pakistan — muzzammilsajid1@gmail.com

**Abstract**—Claims of learned routing superiority can depend on baseline choice, training seed, and test distribution. We therefore compare Dijkstra, A\*, D\* Lite, and Double DQN with Hindsight Experience Replay (DDQN+HER) on one persisted 310-scenario suite with identical movement, obstacle, information, and timing contracts. Seven RL configurations are trained with five policy seeds (35 runs; 17.5 million environment steps) and evaluated on held-out endpoints, layout and dynamics shifts, four grid sizes, and controlled realism factors. Route timings are repeated ten times, and change events are analyzed through cost shock, recovery, reaction work, and post-change success. Across five independently trained policies, DDQN+HER achieved 96.0% mean success on held-out endpoints, 90.0% on unseen matched-density layouts, and 72.0% on denser layouts; A\*, Dijkstra, and D\* Lite achieved 100% on all three splits. RL success fell from 98.0% at  $15 \times 15$  to 0.0% at  $100 \times 100$ , where route decision time was 932.266 ms versus 3.566 ms for A\*. Post-change RL success was 87.9%, despite the recovery criterion being met before termination in all observed events. Ablations were non-monotonic: removing HER, shaping, or curriculum each reached 100% held-out-pair success. All claims are traceable to raw seed/scenario/repetition artifacts.

**Index Terms**—UAV routing, dynamic path planning, D\* Lite, A\*, reinforcement learning, Hindsight Experience Replay, reproducibility

## I. INTRODUCTION

Autonomous UAVs must often route while obstacles, regulatory restrictions, and energy conditions change during flight. A route that is valid at takeoff can become inefficient or infeasible after deployment, making dynamic replanning a central navigation problem [1].

Classical graph search provides strong reference methods when the current map is known. Dijkstra is optimal but expands uniformly; A\* preserves optimality while using a heuristic to focus search; D\* Lite reuses prior search state when edge availability changes [2], [3]. These three algorithms represent meaningfully different classical regimes: naive full replanning, heuristic full replanning, and incremental replanning.

Deep reinforcement learning (RL) offers a different deployment model. After training, a policy selects actions from observations without explicitly solving a graph-search problem at every decision. This may be attractive under rapid changes

or large state spaces, but it removes shortest-path guarantees and introduces sensitivity to training seed, representation, reward design, and distribution shift [4]–[6].

A credible comparison must therefore address more than one policy and one classical baseline. It must pair methods on identical scenarios, retain training and timing distributions, test unseen environments, separate in-distribution from out-of-distribution performance, and measure adaptation after individual change events. This paper implements that evidence standard for a controlled grid-based UAV-routing study.

The study asks four questions:

- 1) How do RL, naive search, heuristic search, and incremental search compare in reliability, path quality, and route-level decision cost?
- 2) How much does RL performance vary across independent training seeds?
- 3) How do the methods respond to layout shift, denser obstacles, new dynamics, realistic cost factors, and increasing grid size?
- 4) Which RL components improve performance under controlled one-factor interventions?

The principal contributions are:

- a persisted 310-scenario benchmark that pairs methods on identical graphs, changes, movement costs, information timing, and compute boundaries;
- a 35-run, 17.5-million-step training study covering five seeds and controlled interventions on Double DQN, HER, reward shaping, curriculum, observation scope, and dynamic-only training;
- seed-aware generalization, scaling, realism, timing, and event-level adaptability analyses with paired tests and multiplicity correction; and
- an auditable pipeline linking source provenance, checkpoints, raw repetitions, tables, vector figures, and manuscript claims.

## II. RELATED WORK

Classical UAV path planning includes graph search, incremental search, sampling-based planning, and potential-field

methods. Dijkstra provides optimal paths for non-negative edge costs [7]. A\* adds an admissible heuristic and normally expands fewer nodes while preserving optimality [2]. D\* Lite reuses earlier search information after edge-cost changes, making it a natural reference for dynamic routing rather than treating repeated full search as the only classical option [3]. Broader UAV-planning surveys emphasize the continued value of completeness, interpretability, and predictable behavior in classical planners [1], [8].

RL methods instead learn a policy through interaction. DQN scales value-based learning with neural function approximation [9]; Double DQN reduces max-operator overestimation [4]; and HER relabels failed trajectories with achieved goals to improve learning in sparse goal-conditioned tasks [5]. UAV studies have applied value-based and deep RL to path planning and obstacle avoidance, reporting adaptability but also dependence on representation, reward, and evaluation conditions [10]–[13]. Recent work also combines temporal observations, recurrent policy learning, and an explicit safety shield for dynamic obstacle avoidance [14]. Those richer sensing and safety assumptions are complementary to this study’s narrower goal: isolating algorithmic behavior under a controlled shared graph and change-event contract.

Evaluation practice is especially important for deep RL. Henderson et al. showed that implementation and random-seed variation can materially alter reported conclusions [6]. A test distribution of many routes from one trained network does not quantify this training variance. Accordingly, this study treats independently trained seeds as the uncertainty units for RL and retains scenario-level pairing within each seed.

Systematic comparisons likewise identify different trade-offs in predictability, adaptation, and computational cost between classical and learned approaches [15]. Prior UAV comparisons often vary the map, metrics, or hardware between methods, which makes algorithmic attribution difficult. This work focuses on controlled same-scenario comparison: all methods receive the same persisted graph and change events, while the benchmark separately varies endpoint, layout, density, dynamics, scale, energy cost, no-fly behavior, and observation noise. The aim is not to claim that a grid benchmark captures complete UAV operation, but to identify precisely where differences arise before moving to richer flight models.

### III. METHODOLOGY

#### A. Shared Routing Model

The operating area is an  $N \times N$  grid with coordinates (*row*, *col*). Movement is eight-connected. Orthogonal edges cost 1 and diagonal edges cost  $\sqrt{2}$ ; diagonal corner cutting is allowed. A destination cell must be inside the grid and traversable. Optional non-negative traversal penalties represent wind, energy demand, or penalized no-fly regions. Hard no-fly cells are removed from the traversable graph. All planners obtain successors and edge costs from the same environment function.

An obstacle may toggle on a fixed period, toggle stochastically under a seeded process, or move along a cyclic path.

Every method follows the same `post_move_observed` information contract: it selects and executes a move from the current state, dynamics then advance, and the changed state is available before the next decision. Terminal moves produce no unobservable event, and methods receive no future schedule. If an obstacle appears in the occupied cell, the benchmark permits the UAV to exit but not re-enter that cell. This legacy contract is shared across methods and reported as a limitation.

#### B. Classical Methods

**Dijkstra** performs a complete shortest-path search initially and after each observed graph change. It is an interpretable full-replanning reference with complexity  $O((V + E) \log V)$  under a binary heap.

**A\*** uses octile distance, the admissible heuristic for the shared eight-connected movement geometry [2]. It shares Dijkstra’s runner and replanning triggers, isolating heuristic pruning as the only algorithmic change.

**D\* Lite** maintains  $g$  and one-step-lookahead  $rhs$  values and updates only vertices affected by changed edges [3]. It receives the same change notifications as the full replanners. Its cumulative expansion count includes the initial computation and all incremental repairs.

#### C. Reinforcement Learning Policy

The learned method is Double DQN [4] with HER [5]. The policy uses a two-layer MLP with 128 units per layer. Its default 61-dimensional observation concatenates normalized UAV position and goal displacement (4), a flattened  $7 \times 7$  local occupancy window (49), and a one-hot previous action (8). The full-observation ablation replaces the local window with the complete grid.

The sparse reward is +1 at the goal, −1 on collision, and −0.1 otherwise. Potential shaping uses

$$F(s, s') = \gamma \Phi(s') - \Phi(s), \quad \Phi(s) = -d_{\text{octile}}(s, g)/d_{\text{max}}, \quad (1)$$

with  $\gamma = 0.99$  [16]. Octile distance is deliberately independent of obstacle state. This is necessary for valid HER relabeling: recomputing a replayed reward from the environment’s obstacle state at sampling time would not reproduce the state under which the transition was collected. Shortest-path distance is retained for evaluation metrics, not used as the dynamic HER potential.

All learned variants use a learning rate of  $10^{-3}$ , replay capacity 100,000, 5,000 learning-start steps, batch size 256, discount 0.99, one gradient update every two environment steps, and a target-network update every 1,000 steps. Exploration decays linearly from 1.0 to 0.05 over the first half of training. HER samples four achieved goals per transition. These values are fixed across variants unless the named intervention removes a component.

For each policy seed, training proceeds through 300,000 static steps on the fixed density-0.20 layout, 100,000 steps with one mild dynamic obstacle on the same-density base layout, and 100,000 steps with the full fixed-toggle configuration on that same base layout. The environment layout seed is held at

42 while policy seeds vary, separating optimization variance from layout variance. Checkpoints, logs, exact configuration, throughput, and machine metadata are stored for each seed. The policy seeds are 11, 22, 33, 44, and 55. Seven configurations times five seeds yield 35 completed runs and 17.5 million environment interactions.

#### D. Controlled Ablations

The complete method is compared with vanilla DQN, HER disabled, potential shaping disabled, curriculum disabled, full-grid observation, and training on the full dynamic condition from scratch. Each ablation changes one named factor. Dynamic-from-scratch training receives 500,000 dynamic steps so its interaction budget matches the complete curriculum’s cumulative budget.

#### E. Adaptability Metrics

For every observed change event, we record:

- the optimal remaining cost immediately before and after the change;
- *optimal-cost shock*, their difference;
- *recovery steps*, the decisions until optimal remaining cost returns to or below its pre-change value;
- event reaction time and node expansions for classical planners, or policy decision latency for RL; and
- whether the route succeeds after the event.

Unrecovered events remain explicitly missing rather than being dropped.

### IV. EXPERIMENTAL SETUP

#### A. Persisted Benchmark Splits

The benchmark manifest contains 310 scenarios. Every record stores its scenario ID, split, grid size, exact blocked cells, start and goal, dynamic objects, traversal penalties, no-fly configuration, observation-noise level, and dynamics seed where applicable.

The primary generalization splits each contain 30 scenarios:

- the training layout with held-out start/goal pairs;
- unseen layouts at obstacle density 0.20;
- unseen layouts at density 0.30;
- unseen fixed-toggle obstacle locations; and
- known toggle locations with unseen periods.

Scaling uses ten scenarios each at 15, 30, 50, and 100 cells per side, with density 0.20 and new layouts. Realism sweeps independently vary static density, stochastic toggle probability, moving-obstacle period, spatial energy penalty, hard versus penalized no-fly zones, and sensor-noise probability.

#### B. Pairing and Repetitions

All methods consume the same manifest records. The grid is reconstructed from stored cells rather than from a method-specific random generator. Classical planners run on the same graph instance definition; the RL evaluator injects the identical grid and dynamics into its Gymnasium environment.

Each classical scenario/planner pair and each RL seed/scenario pair is repeated ten times for timing. Outcome

and path metrics are deterministic under a fixed scenario and policy; repetitions preserve the raw timing distribution. Route-level compute time is the sum of planner calls for classical methods and the sum of `predict()` calls for RL. Environment construction, stepping, metric calculation, and serialization are excluded. Per-decision latency is also retained. Local-observation policies cover all 310 scenarios, producing 15,500 raw rows per variant ( $310 \times 5 \times 10$ ). The fixed-size full-grid ablation is evaluated only at its trained  $15 \times 15$  input dimension, producing 14,000 rows ( $280 \times 5 \times 10$ ); this exclusion is structural rather than a performance-based omission.

#### C. Environment and Software Manifest

The authoritative classical and RL suite manifests both report Linux 6.6.122+, CPython 3.12.13, two logical CPU cores, and no active GPU. Every manifest stores operating system, CPU, logical core count, Python implementation and version, package versions, accelerator availability and names, Git commit, command configuration, manifest hash, and timestamp. The raw suite was produced in a Drive-backed Google Colab workflow. Millisecond timings are interpreted only within this recorded runtime class. The dedicated timing-distribution artifact reports the mean, median, sample standard deviation, extrema, and repetition count for route-level computation and per-decision latency, both by policy seed and across seeds.

Historically, one mutable “latest RL” environment file was retained rather than an immutable manifest for every variant; the artifact pipeline now writes both variant-specific manifests and the compatibility alias. Consequently, absolute cross-variant timing differences are descriptive, whereas same-scenario comparisons within a recorded suite are the primary evidence.

#### D. Statistical Analysis

Timing repetitions are first collapsed within each policy-seed/scenario pair, preventing pseudoreplication. RL metrics are then summarized per independent training seed and reported as the mean and 95% Student-*t* confidence interval across five seeds. Classical confidence intervals use scenarios as the independent units.

Success differences use exact paired McNemar tests. Path cost and route compute time use two-sided paired Wilcoxon signed-rank tests with rank-biserial effect sizes [17], [18]. Only jointly successful routes enter paired path-cost tests; all routes enter success and compute-time analyses. Holm correction controls the familywise error rate across the reported paired tests [19].

#### E. Reproducibility Checks

Before evaluation, all persisted scenarios must reconstruct with a valid path and Dijkstra, A\*, and D\* Lite must agree on the initial optimal cost. After evaluation, an integrity pass checks expected row counts, unique run IDs, and the scenario-manifest hash in every raw result file. Smoke-test checkpoints and legacy single-seed models are rejected by the expanded evaluator.

## V. RESULTS

Table I separates endpoint generalization from layout and dynamics shifts. The full RL policy changes from 96.0% success on the training layout with held-out endpoints to 90.0% on unseen matched-density layouts and 72.0% on denser layouts. The three classical planners remain at 100% on these splits, so the experiment provides no reliability advantage for the learned method. Scaling exposes the strongest boundary: RL success is 98.0%, 56.0%, 14.0%, and 0.0% at grid widths 15, 30, 50, and 100, while every classical planner remains at 100%. At  $100 \times 100$ , the full RL policy uses 932.266 ms of route decision time versus 3.566 ms for A\*. Thus, these implementations show no scale crossover in favor of RL.

The held-out-pair ablation in Table IV is also non-monotonic. Vanilla DQN and the full-grid observation reach only 38.0% and 15.3%, respectively, supporting Double DQN and the local observation in this setup. Yet removing HER (100.0%), shaping (100.0%), or the curriculum (100.0%) each exceeds the full method’s 96.0% on this split. These one-factor interventions show configuration sensitivity, not separable causal main effects, because the learning components interact.

At matched change events, the full RL method attains 87.9% post-change success, while its recovery criterion is met in 100.0% of observed events. This distinction matters: recovery of optimal remaining cost before termination does not guarantee that the complete route succeeds. The corresponding D\* Lite values are 100.0% and 100.0%.

### A. Generalization

TABLE I

GENERALIZATION SUMMARY (MEAN  $\pm$  95% CI). C SUCC. IS THE MINIMUM SUCCESS AMONG A\*, DIJKSTRA, AND D\* LITE; RL ESTIMATES USE FIVE POLICIES AND GAP USES SUCCESSFUL ROUTES.

| Condition          | C succ. (%) | RL succ. (%)    | RL gap          | A* ms           | RL ms            |
|--------------------|-------------|-----------------|-----------------|-----------------|------------------|
| Held-out pairs     | 100.0       | 96.0 $\pm$ 11.1 | 1.11 $\pm$ 0.45 | 0.15 $\pm$ 0.02 | 6.61 $\pm$ 9.77  |
| Unseen layout      | 100.0       | 90.0 $\pm$ 5.9  | 0.90 $\pm$ 0.38 | 0.16 $\pm$ 0.04 | 11.16 $\pm$ 3.79 |
| Denser layout      | 100.0       | 72.0 $\pm$ 8.6  | 0.92 $\pm$ 0.38 | 0.18 $\pm$ 0.04 | 26.32 $\pm$ 6.95 |
| New obstacle sites | 100.0       | 98.0 $\pm$ 5.6  | 0.93 $\pm$ 0.17 | 0.20 $\pm$ 0.05 | 4.59 $\pm$ 2.86  |
| New toggle periods | 100.0       | 98.7 $\pm$ 3.7  | 0.88 $\pm$ 0.35 | 0.48 $\pm$ 0.14 | 4.86 $\pm$ 3.28  |

### B. Scaling

TABLE II

SCALING SUMMARY (MEAN  $\pm$  95% CI). C SUCC. IS THE MINIMUM SUCCESS AMONG A\*, DIJKSTRA, AND D\* LITE; RL ESTIMATES USE FIVE POLICIES AND GAP USES SUCCESSFUL ROUTES.

| Condition        | C succ. (%) | RL succ. (%)    | RL gap          | A* ms           | RL ms                |
|------------------|-------------|-----------------|-----------------|-----------------|----------------------|
| 15 $\times$ 15   | 100.0       | 98.0 $\pm$ 5.6  | 1.57 $\pm$ 0.48 | 0.32 $\pm$ 0.12 | 4.39 $\pm$ 1.09      |
| 30 $\times$ 30   | 100.0       | 56.0 $\pm$ 28.6 | 2.89 $\pm$ 2.21 | 0.51 $\pm$ 0.21 | 115.57 $\pm$ 94.04   |
| 50 $\times$ 50   | 100.0       | 14.0 $\pm$ 16.7 | 4.10 $\pm$ 9.01 | 0.98 $\pm$ 0.61 | 485.83 $\pm$ 221.62  |
| 100 $\times$ 100 | 100.0       | 0.0 $\pm$ 0.0   | –               | 3.57 $\pm$ 2.05 | 932.27 $\pm$ 1103.25 |

### C. Controlled Realism Factors

TABLE III

CONTROLLED REALISM SUMMARY (MEAN  $\pm$  95% CI). C SUCC. IS THE MINIMUM SUCCESS AMONG THE THREE CLASSICAL PLANNERS; RL GAP USES SUCCESSFUL ROUTES.

| Condition                | C succ. (%) | RL succ. (%)    | RL gap          | A* ms           | RL ms             |
|--------------------------|-------------|-----------------|-----------------|-----------------|-------------------|
| Static density 0.40      | 100.0       | 72.0 $\pm$ 26.9 | 1.32 $\pm$ 0.45 | 0.19 $\pm$ 0.05 | 26.29 $\pm$ 20.59 |
| Stochastic toggle 0.20   | 100.0       | 100.0 $\pm$ 0.0 | 1.18 $\pm$ 0.45 | 0.39 $\pm$ 0.15 | 4.11 $\pm$ 0.69   |
| Moving obstacle, $T = 2$ | 100.0       | 100.0 $\pm$ 0.0 | 1.32 $\pm$ 0.33 | 0.66 $\pm$ 0.23 | 5.82 $\pm$ 0.93   |
| Energy penalty 1.0       | 100.0       | 100.0 $\pm$ 0.0 | 1.98 $\pm$ 0.54 | 0.15 $\pm$ 0.04 | 2.93 $\pm$ 0.27   |
| Hard no-fly              | 100.0       | 78.0 $\pm$ 26.9 | 1.58 $\pm$ 0.40 | 0.17 $\pm$ 0.08 | 16.55 $\pm$ 14.76 |
| Penalized no-fly         | 100.0       | 92.0 $\pm$ 16.2 | 1.75 $\pm$ 0.53 | 0.33 $\pm$ 0.14 | 10.79 $\pm$ 14.32 |
| Sensor noise 0.10        | 100.0       | 100.0 $\pm$ 0.0 | 1.44 $\pm$ 0.37 | 0.31 $\pm$ 0.11 | 3.77 $\pm$ 0.64   |

### D. RL Ablations

TABLE IV

HELD-OUT-PAIR ABLATIONS ACROSS FIVE POLICY SEEDS (MEAN  $\pm$  95% CI); AN INTERVAL IS OMITTED WHEN FEWER THAN TWO FINITE SEED ESTIMATES ARE AVAILABLE.

| Variant            | Success (%)     | Cost gap        | Route ms          |
|--------------------|-----------------|-----------------|-------------------|
| DQN+HER            | 38.0 $\pm$ 63.0 | 0.63 $\pm$ 1.39 | 53.00 $\pm$ 55.69 |
| Dynamic-only train | 92.0 $\pm$ 20.0 | 1.13 $\pm$ 0.43 | 8.20 $\pm$ 11.63  |
| DDQN+HER           | 96.0 $\pm$ 11.1 | 1.11 $\pm$ 0.45 | 6.61 $\pm$ 9.77   |
| Full-grid obs.     | 15.3 $\pm$ 42.6 | 3.18            | 13.84 $\pm$ 5.93  |
| No curriculum      | 100.0 $\pm$ 0.0 | 0.87 $\pm$ 0.16 | 3.55 $\pm$ 0.71   |
| No HER             | 100.0 $\pm$ 0.0 | 1.01 $\pm$ 0.14 | 3.47 $\pm$ 0.69   |
| No shaping         | 100.0 $\pm$ 0.0 | 0.92 $\pm$ 0.41 | 3.21 $\pm$ 0.50   |

### E. Event-Level Adaptability

TABLE V

EVENT-LEVEL ADAPTABILITY ACROSS ALL DYNAMIC SCENARIOS. VALUES ARE MEANS AND 95% CONFIDENCE INTERVALS OVER INDEPENDENT SCENARIOS FOR CLASSICAL PLANNERS AND POLICY SEEDS FOR RL.

| Method   | $n$ | Post-success (%) | Recovery (%)    | Cost shock        | Reaction ms       |
|----------|-----|------------------|-----------------|-------------------|-------------------|
| A*       | 83  | 100.0 $\pm$ 0.0  | 100.0 $\pm$ 0.0 | 0.000 $\pm$ 0.000 | 0.067 $\pm$ 0.007 |
| Dijkstra | 83  | 100.0 $\pm$ 0.0  | 100.0 $\pm$ 0.0 | 0.001 $\pm$ 0.002 | 0.442 $\pm$ 0.062 |
| D* Lite  | 83  | 100.0 $\pm$ 0.0  | 100.0 $\pm$ 0.0 | 0.005 $\pm$ 0.008 | 0.168 $\pm$ 0.018 |
| DDQN+HER | 5   | 87.9 $\pm$ 33.6  | 100.0 $\pm$ 0.0 | 0.005 $\pm$ 0.002 | 0.521 $\pm$ 0.050 |

The dedicated event files report optimal-cost shock, positive extra optimal cost attributable to the change, recovery steps, post-change success, reaction time, and search expansions for every change. Events that do not recover before termination remain in the denominator and retain a missing recovery time. This avoids the optimistic bias that would result from analyzing only recovered or successful events.

### F. Paired Statistical Tests

The machine-readable paired-test artifact reports exact McNemar tests for success and Wilcoxon signed-rank tests for cost and compute time, together with rank-biserial effect sizes and Holm-adjusted  $p$ -values. Seed-specific RL comparisons remain separate in the raw statistical output; aggregate confidence intervals are computed from seed-level estimates rather than treating routes from one policy as independent trained agents. A second paired-test artifact applies the same controls to matched change events, using change step and changed-cell

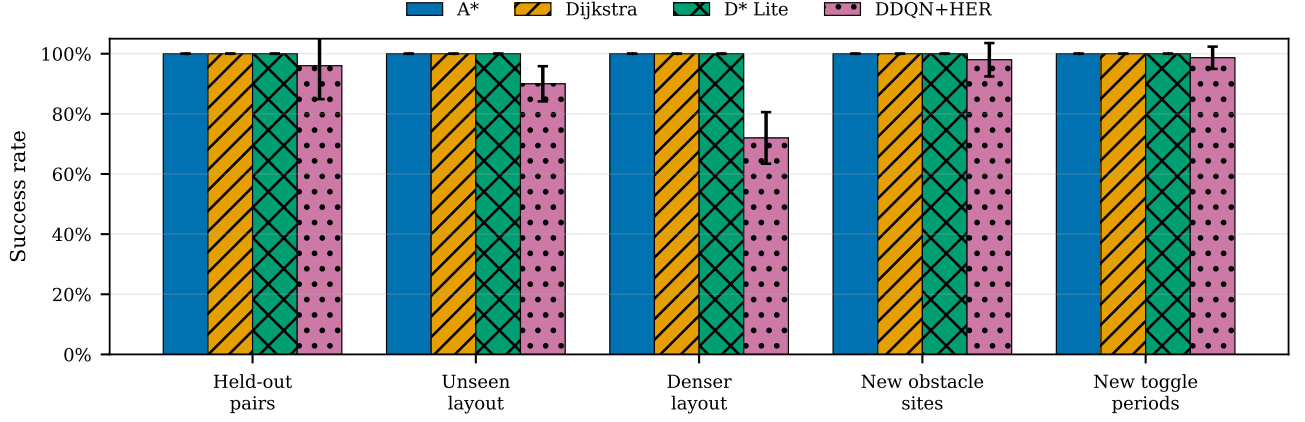


Fig. 1. Generalization under paired benchmark conditions. Classical bars aggregate scenarios; DDQN+HER error bars are 95% confidence intervals across five independently trained policies. Hatches preserve distinctions in grayscale.

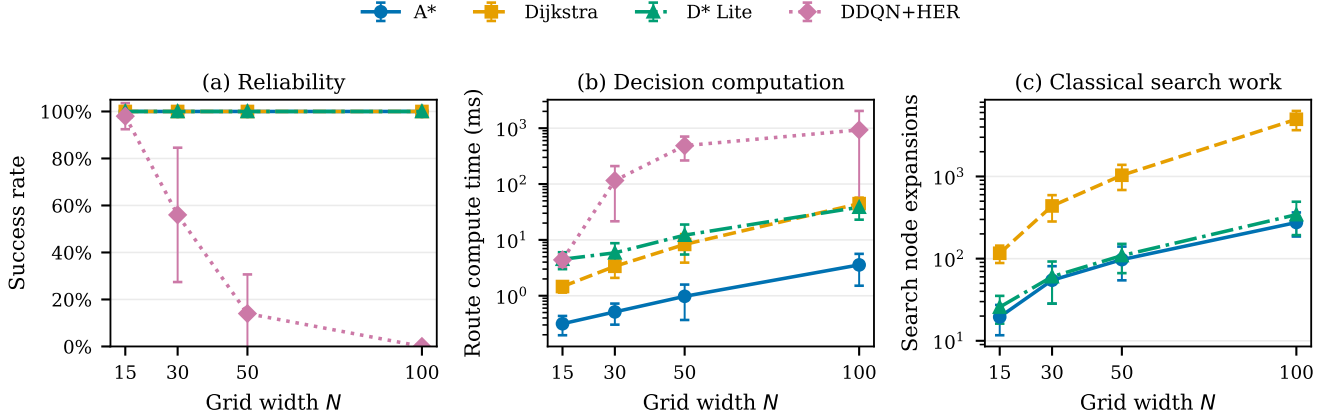


Fig. 2. Scaling behavior: (a) reliability, (b) route-level decision computation on a logarithmic axis, and (c) classical search work on a logarithmic axis. Model-free RL has no search-node expansion count.

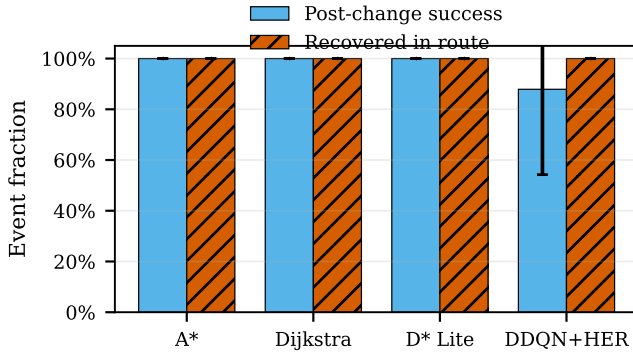


Fig. 3. Post-change route success and recovery before termination across all dynamic scenarios.

signature rather than event order alone. The exhaustive family contains 13,014 route-level tests and is deliberately conserva-

tive. As one concrete effect, A\* is faster than DDQN+HER on every denser-layout seed comparison (Wilcoxon rank-biserial effect = 1.0; globally Holm-adjusted  $p = 2.24 \times 10^{-5}$  for each seed). Success and cost effects should be read with their confidence intervals and adjusted tests, not with uncorrected route-level significance.

## VI. DISCUSSION

This study is designed to distinguish three questions that a single Dijkstra-versus-policy comparison cannot answer: whether heuristic pruning changes the classical compute baseline, whether incremental reuse changes it again, and whether RL behavior is stable across training seeds and distribution shifts. The generalization, scaling, realism, and ablation tables should therefore be interpreted together rather than reducing performance to one headline average.

Classical methods retain an important structural advantage: on the currently observed graph they provide an optimal route, and failures arise from future unobserved changes or route

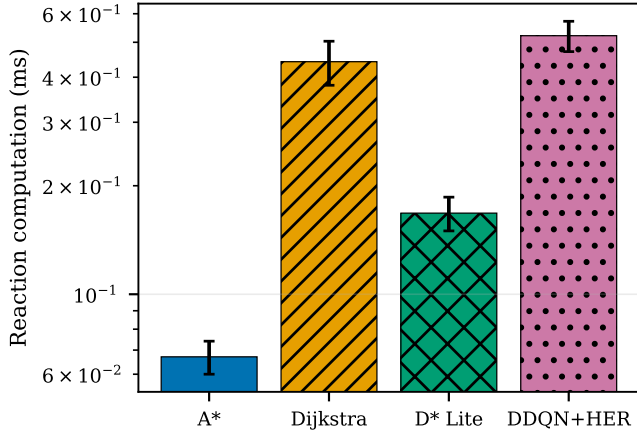


Fig. 4. Event reaction computation on a logarithmic scale. Classical values time the repair; RL values time the first post-change decision.

semantics rather than approximation by the search itself. A\* measures how far simple heuristic pruning goes; D\* Lite measures the additional value of reusing prior search effort. The measured data do not show a deployment-time crossover in favor of RL. At  $100 \times 100$ , every classical method succeeds on every scenario, while the learned policy succeeds on none and is slower than A\*. The defensible scope for this policy is therefore the native small grid and closely related layouts, not scale-independent routing.

The multi-seed design changes how learned performance is interpreted. Scenario confidence intervals from one frozen policy quantify test-case variation, not training variance. Reporting seed-level values first ensures the uncertainty shown for RL reflects independently learned policies. Likewise, the ablations reveal interactions rather than a tidy component ranking. Double DQN and local observations improve markedly over vanilla DQN and full-grid input, but removing HER, shaping, or curriculum improves held-out-pair success from 96% to 100% under this protocol. A one-factor intervention demonstrates configuration sensitivity; in a nonlinear learning system it does not isolate a context-free causal main effect.

#### A. Limitations

The environment remains a two-dimensional discrete grid rather than a continuous flight-dynamics simulator. Movement ignores vehicle turn radius, altitude, acceleration, localization drift, communication constraints, and multi-UAV interactions. Wind and energy are represented as spatial traversal penalties rather than aerodynamic forces. The study therefore supports claims about routing algorithms under controlled graph changes, not end-to-end flight control.

The legacy contract permits departure from a cell that becomes blocked while occupied. This avoids instantaneous failures caused by a toggle and is applied equally to all methods, but it is only one possible physical interpretation. Future work should compare escape, collision, and safety-buffer semantics.

Training uses one fixed static layout before dynamic fine-tuning. The benchmark measures the resulting distribution shift explicitly, but does not establish the performance of policies trained over randomized layout distributions. The full-observation ablation is dimension-specific and cannot be applied unchanged to larger grids, unlike the fixed-size local observation.

Only five independent policy seeds are available. Student- $t$  intervals expose the resulting uncertainty, but remain wide for unstable variants and cannot characterize rare optimization failures as precisely as a larger seed study. The 310-scenario suite is broad within the grid model but is not a random sample of all operational UAV environments.

Finally, timings describe these Python and PyTorch implementations on the recorded Colab runtime class. Historical per-variant machine manifests were not retained separately, so absolute cross-variant timing contrasts warrant extra caution. The pipeline now prevents that provenance loss. Timings remain auditable implementation-specific measurements, not hardware-independent constants; operational conclusions require optimized code and embedded target hardware.

## VII. CONCLUSION

This paper replaces a single-policy, single-baseline comparison with a paired, multi-seed evaluation of full, heuristic, incremental, and learned replanning. The evidence spans 310 persisted scenarios, 35 training runs, 17.5 million environment steps, repeated route timings, controlled distribution shifts, and matched change events.

Under the shared graph, timing, and information contracts, the classical planners provide the strongest reliability and scale in this benchmark. DDQN+HER is competitive only on the native small-grid setting: it achieves 96.0% held-out-pair success and 90.0% on unseen matched-density layouts, but falls to 0.0% at  $100 \times 100$  while all classical planners remain at 100%. The ablations further reject a simple component story: Double DQN and local observations help, whereas HER, shaping, and curriculum are not individually supported as beneficial under the tested protocol. The contribution is therefore not a claim of RL superiority, but a reproducible map of where learned routing works, where it fails, and which conclusions survive seed variation, paired scenarios, distribution shift, scale, and event-level analysis.

## APPENDIX A ARTIFACT TRACEABILITY

The following map links each result family to its executable command, persisted scenario source, raw evidence, and generated manuscript artifact. Every expanded-suite raw row stores the SHA-256 digest of `benchmark_v2.json`; the integrity checker rejects unexpected row counts, duplicate run identifiers, and digest mismatches. The PDF build manifest additionally records the integrity-report digest, Tectonic version, page count, and final PDF digest.

- **Classical pilot:** `scripts/reproduce_classical.py` maps the Week-3 manifest and `classical_dynamic_`

raw.csv to its machine-readable and Markdown summaries.

- **Generalization, scaling, and realism:** scripts/run\_full\_research.py maps benchmark\_v2.json, classical\_suite\_raw.csv, and each rl\_<variant>\_raw.csv to Tables I–III and the scaling figures.
- **RL ablations:** the same command with --train maps variant-specific checkpoints and raw CSVs to Table IV.
- **Adaptability:** evaluation/analyze\_adaptability.py maps the route event CSVs to Table V, paired event tests, and the recovery and reaction-time figures.
- **Statistics:** evaluation/statistical\_tests.py and the adaptability analyzer produce paired\_tests.csv and adaptability\_paired\_tests.csv with effect sizes and adjusted  $p$ -values. evaluation/summarize\_timing\_distributions.py produces the seed-specific and aggregate timing distribution artifact.

## REFERENCES

- [1] S. Ghambari, M. Golabi, L. Jourdan, J. Lepagnot, and L. Idoumghar, “Uav path planning techniques: a survey,” *RAIRO - Operations Research*, vol. 58, no. 4, pp. 2951–2989, 2024.
- [2] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [3] S. Koenig and M. Likhachev, “Fast replanning for navigation in unknown terrain,” *IEEE Transactions on Robotics*, vol. 21, no. 3, pp. 354–363, 2005.
- [4] H. van Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double q-learning,” in *Proceedings of the 30th AAAI Conference on Artificial Intelligence*, 2016, pp. 2094–2100.
- [5] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, P. Abbeel, and W. Zaremba, “Hindsight experience replay,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [6] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, “Deep reinforcement learning that matters,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, pp. 3207–3214, 2018.
- [7] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.
- [8] S. Venu and M. Gurusamy, “A comprehensive review of path planning algorithms for autonomous navigation,” *Results in Engineering*, vol. 28, p. 107750, 2025.
- [9] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [10] G.-T. Tu and J.-G. Juang, “Uav path planning and obstacle avoidance based on reinforcement learning in 3d environments,” *Actuators*, vol. 12, no. 2, p. 57, 2023.
- [11] Y. Guo and Z. Liu, “Uav path planning based on deep reinforcement learning,” *International Journal of Advanced Network, Monitoring and Controls*, vol. 8, no. 3, pp. 81–88, 2023.
- [12] J. Zhao, Z. Gan, J. Liang, C. Wang, K. Yue, W. Li, Y. Li, and R. Li, “Path planning research of a uav base station searching for disaster victims’ location information based on deep reinforcement learning,” *Entropy*, vol. 24, no. 12, p. 1767, 2022.
- [13] L. G. S. d. Rocha, K. A. Q. Caldas, M. H. Terra, F. Ramos, and K. C. T. Vivaldini, “Dynamic q-planning for online uav path planning in unknown and complex environments,” 2024.
- [14] C. Liu and S. Wang, “Temporally-aware deep reinforcement learning for dynamic obstacle avoidance in uavs,” *Drones*, vol. 10, no. 7, p. 505, 2026.
- [15] A. Mannan, M. S. Obaidat, K. Mahmood, A. Ahmad, and R. Ahmad, “Classical versus reinforcement learning algorithms for unmanned aerial vehicle network communication and coverage path planning: A systematic literature review,” *International Journal of Communication Systems*, vol. 36, no. 5, p. e5423, 2023.
- [16] A. Y. Ng, D. Harada, and S. Russell, “Policy invariance under reward transformations: Theory and application to reward shaping,” in *Proceedings of the 16th International Conference on Machine Learning (ICML)*, 1999, pp. 278–287.
- [17] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
- [18] F. Wilcoxon, “Individual comparisons by ranking methods,” *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.
- [19] S. Holm, “A simple sequentially rejective multiple test procedure,” *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.